ADBMS Project: Full Implementation Guide (Rebalanced Distribution)

Supermarket chain system | MySQL 8 + Spring Boot (JDBC, no ORM) + Angular | ICT3222 Rebalanced Workload Edition: Guaranteed equal ADBMS rubric coverage for all 5 members

1. Goal and scope

Build a multi-branch supermarket system where **the database does the real work** (procedures, triggers, events, views, functions, indexes, locking, and roles) and the application is a thin layer that calls it. Every required curriculum topic maps directly to concrete features across all 5 team members.

Required topic	Where it appears	Primary Member Owner
Primary index	PK on every table (section 5)	All members
Clustering index	InnoDB clusters rows by PK; composite PKs on `order_item`, `inventory`, `daily_sales_summary` (6.1)	Member 1, Member 2, Member 4
Secondary index	Indexes on date, branch, phone, name; EXPLAIN before/after benchmarks (6.1)	Member 3, Member 4
Stored procedures	`sp_place_order`, `sp_cancel_order`, `sp_add_stock`, `sp_transfer_stock`, `sp_apply_discounts`, `sp_monthly_report`, `sp_deactivate_employee` (6.4, 6.8)	Member 1, 2, 3, 4, 5
Functions (UDF)	`fn_order_total`, `fn_customer_tier`, `fn_stock_level`, `fn_avg_daily_sales`, `fn_mask_phone`, `fn_is_holiday`, `fn_coupon_valid` (6.3, 6.7)	Member 1, 2, 3, 4, 5
Triggers	Stock check & deduction, low-stock alert, price audit, discount rule check & audit, employee audit (6.5, 6.8)	Member 1, 2, 3, 4, 5
Events	Daily summary, weekly low-stock scan, monthly log purge, nightly discount expiry (6.6, 6.8)	Member 2, 3, 4
Views	Daily sales, low stock, customer orders, product ranking (window function), active discounts, discount impact, masked customer directory (6.2, 6.8, 6.9)	Member 1, 2, 3, 4, 5
Transactions & ACID	Atomic order placement with rollback inside `sp_place_order`, `sp_cancel_order`, `sp_transfer_stock`, `sp_deactivate_employee`	Member 1, 2, 5
Business rules in DB	Multi-tier discount engine: cursor, functions, triggers, events, views (6.7, 6.8)	Member 3 (with Member 1 integration)
Locking & Concurrency	`SELECT ... FOR UPDATE` on inventory and customer rows; deadlock retry handling (6.8, 6.9)	Member 1, Member 2
Security & Roles	MySQL 8 `CREATE ROLE`, dynamic data masking, least-privilege DB users (6.9, 7)	Member 5

2. Tech stack

Layer	Choice	Notes
Database	MySQL 8.0 (InnoDB)	Events, `JSON_TABLE`, window functions, roles, CTEs available
Backend	Spring Boot 3, Java 17, Maven	Dependencies: Web, **JDBC API**, MySQL Driver, Security, Validation
DB access	Spring JDBC (`JdbcTemplate` / `SimpleJdbcCall`)	**Strictly No JPA, No Hibernate, No @Entity**
Auth	Spring Security + JWT, BCrypt	Roles: `ADMIN`, `MANAGER`, `CASHIER`
Frontend	Angular 17+ with Angular Material	Role-based routes, JWT interceptor, standalone components
Tools	MySQL Workbench, Postman, draw.io, GitHub	Keep all `.sql` scripts versioned in repository
Dummy data	Python Faker / recursive script	50k+ orders so index benchmarks and EXPLAIN are authentic
Hosting (demo)	Localhost / Docker Compose	Justify AWS RDS / Azure Database for MySQL in the report

3. Architecture

```
Angular (4200) --HTTPS/JSON+JWT--> Spring Boot (8080)
   Controller -> Service -> DAO (JdbcTemplate)
                                 |
                                 | JDBC (HikariCP pool, SSL)
                                 v
                    MySQL 8: tables + indexes
                    procedures | functions | triggers | views | events | roles
```

Golden Rule:

- **Controller** only parses HTTP and validates input.
- **Service** coordinates workflows (no domain math).
- **DAO** is the only place SQL lives and exclusively executes `CALL procedure(...)` or `SELECT ... FROM view`.
- **Database** enforces all rules, calculations, constraints, audit logging, and locking.

3A. Repository layout: one monorepo

```
supermarket-adbms/
├── README.md                          # Run guide + team allocation table
├── docker-compose.yml                 # mysql + backend + frontend orchestration
├── .env.example                       # DB_USER, DB_PASS, JWT_SECRET template
├── .gitignore                         # target/, node_modules/, .env, *.log
├── docs/                              # ER diagram, report drafts, EXPLAIN screenshots
│
├── database/                          # ALL SQL scripts in numbered execution order
│   ├── 01_schema.sql                  # Core tables & foreign keys
│   ├── 02_schema_discount.sql         # Holiday, discount_rule, coupon, order_discount
│   ├── 02_schema_security.sql         # Employee audit and security logs (Member 5)
│   ├── 03_indexes.sql                 # Secondary, composite & fulltext indexes
│   ├── 04_functions.sql               # Scalar UDFs (totals, tiers, masking, stock)
│   ├── 05_procedures_sales.sql        # Member 1: sp_place_order, sp_cancel_order
│   ├── 05_procedures_inventory.sql    # Member 2: sp_add_stock, sp_receive_purchase, sp_transfer_stock
│   ├── 05_procedures_products.sql     # Member 3: sp_save_product, sp_register_customer, sp_apply_discounts
│   ├── 05_procedures_reports.sql      # Member 4: sp_monthly_report, sp_category_sales_cte
│   ├── 05_procedures_security.sql     # Member 5: sp_get_login, sp_deactivate_employee, sp_change_user_role
│   ├── 06_triggers.sql                # Stock checks, price audit, rule check, employee audit
│   ├── 07_views.sql                   # Analytical, ranking, and data masking views
│   ├── 08_events.sql                  # Daily summaries, scans, rule expiry, log purging
│   ├── 09_roles_users.sql             # MySQL 8 roles, app users, and grants (Member 5)
│   ├── 10_seed_reference.sql          # Reference branches, categories, products, rules
│   ├── seed_bulk/generate_orders.py   # 50,000+ orders generator (Member 4)
│   └── tests/                         # 5 test scripts (one per member module)
│
├── backend/                           # Spring Boot (Maven)
│   ├── pom.xml                        # Strict JDBC dependencies (No JPA)
│   ├── mvnw, mvnw.cmd, .mvn/          # Maven wrapper
│   ├── Dockerfile
│   └── src/main/java/lk/supermarket/
│       ├── config/                    # DataSourceConfig, SecurityConfig, CorsConfig (Member 5)
│       ├── common/                    # GlobalExceptionHandler, ApiError, JwtUtil (Member 5)
│       ├── sales/                     # OrderController, OrderService, OrderDao (Member 1)
│       ├── inventory/                 # InventoryController, PurchaseController, InventoryDao (Member 2)
│       ├── product/                   # ProductController, CategoryController, ProductDao (Member 3)
│       ├── customer/                  # CustomerController, CustomerService, CustomerDao (Member 3)
│       ├── discount/                  # DiscountController, CouponController, DiscountDao (Member 3)
│       ├── report/                    # ReportController, ReportService, ReportDao (Member 4)
│       ├── auth/                      # AuthController, AuthService, AuthDao (Member 5)
│       └── audit/                     # AuditController, AuditService, AuditDao (Member 5)
│
└── frontend/                          # Angular 17+ (npm)
    ├── package.json, angular.json
    ├── proxy.conf.json                # Dev proxy forwarding /api to localhost:8080
    ├── Dockerfile, nginx.conf
    └── src/app/
        ├── core/                      # Auth service, JWT interceptor, Role guards (Member 5)
        ├── shared/                    # Navbar, dialogs, pipes, models (Member 1)
        └── features/
            ├── pos/                   # Member 1: POS screen, live cart, receipt
            ├── inventory/             # Member 2: Stock tables, restock, transfer, POs
            ├── products/              # Member 3: Catalog CRUD, search, category filter
            ├── customers/             # Member 3: Customer registration, tier display
            ├── discounts/             # Member 3: Discount rules, coupons, holiday calendar
            ├── reports/               # Member 4: Charts, monthly sales, top ranking
            ├── auth/login/            # Member 5: Login view
            └── admin/                 # Member 5: Employee admin, security logs, data masking
```

---

4. Modules to implement

#	Module	What it does	Key DB Objects	Primary Owner
M1	POS & Sales	Atomic checkout, inventory deduction, order receipt, cancellation	`sp_place_order`, `sp_cancel_order`, `trg_orderitem_check_stock`, `trg_orderitem_reduce_stock`	**Member 1**
M2	Inventory & Supply	Stock levels, purchase orders, restock, branch-to-branch transfer	`inventory`, `sp_add_stock`, `sp_receive_purchase`, `sp_transfer_stock`, `trg_inventory_low_stock`, `v_low_stock`	**Member 2**
M3	Catalog & Pricing	Product catalog, supplier mapping, price change audit	`product`, `category`, `sp_save_product`, `trg_product_price_audit`, `FULLTEXT` index	**Member 3**
M4	Customer & Loyalty	Customer registration, phone index, tier classification	`customer`, `sp_register_customer`, `fn_customer_tier`, `v_customer_orders`	**Member 3**

M5	Discounts & Rules	First-purchase, holiday, tier, and coupon rules evaluated via Cursor	`discount_rule`, `coupon`, `sp_apply_discounts`, `trg_rule_check`, `trg_rule_audit`, `ev_expire_discount_rules`	**Member 3**
M6	Reports & Analytics	Daily/monthly trends, window functions, rollup aggregations	`v_daily_sales`, `v_product_ranking`, `sp_monthly_report`, `ev_daily_sales_summary`, `v_discount_impact`	**Member 4**
M7	Query Optimization	50k+ bulk seed data, indexing benchmarks, table partitioning	`generate_orders.py`, partitioning on `stock_audit_log`, `EXPLAIN ANALYZE` benchmarks	**Member 4**
M8	Security & RBAC	MySQL 8 roles, database users, access control, login verification	`cashier_role`, `manager_role`, `admin_role`, `sp_get_login`, dedicated DB accounts	**Member 5**
M9	Admin & Audit	Safe employee management, employee audit trigger, data masking	`sp_deactivate_employee`, `trg_employee_audit`, `fn_mask_phone`, `v_masked_customer_directory`	**Member 5**

---

5. Database structure

5.1 Tables

Table	Columns (PK in bold)	Purpose
`branch`	**branch_id**, name, city, province, address, phone	Supermarket chain branches
`employee`	**employee_id**, branch_id (FK), full_name, role ENUM, username (UNIQUE), password_hash, hire_date, is_active	System staff and users
`customer`	**customer_id**, full_name, phone (UNIQUE), email, loyalty_points, registered_date	Shoppers
`category`	**category_id**, name (UNIQUE), parent_id (FK, nullable)	Category tree
`supplier`	**supplier_id**, name, phone, email, city	Wholesale vendors
`product`	**product_id**, category_id (FK), supplier_id (FK), sku (UNIQUE), name, unit_price, cost_price, is_active	Product catalog
`inventory`	**(branch_id, product_id)** composite PK, quantity, reorder_level, version INT DEFAULT 0	Per-branch stock with optimistic locking
`sales_order`	**order_id**, branch_id (FK), customer_id (FK, nullable), employee_id (FK), order_date, status ENUM, subtotal, discount_total, total_amount	Sales transactions
`order_item`	**(order_id, product_id)** composite PK, quantity, unit_price, line_total (generated column)	Order line items clustered together
`payment`	**payment_id**, order_id (FK), method ENUM, amount, paid_at	Order payment records
`purchase_order`	**po_id**, supplier_id (FK), branch_id (FK), order_date, status ENUM	Vendor restock orders
`purchase_order_item`	**(po_id, product_id)**, quantity, unit_cost	PO items
`price_audit_log`	**log_id**, product_id, old_price, new_price, changed_by, changed_at	Immutable price update log
`stock_audit_log`	**log_id**, branch_id, product_id, change_qty, reason, ref_order_id, logged_at	Partitioned stock change history
`low_stock_alert`	**alert_id**, branch_id, product_id, quantity, created_at, resolved (BOOL)	Stock reorder warnings
`daily_sales_summary`	**(summary_date, branch_id)** composite PK, total_orders, total_sales	Precomputed daily totals
`holiday`	**holiday_date**, name	National holiday calendar
`discount_rule`	**rule_id**, name, rule_type, kind, value, min_order_amount, max_discount, tier, start_date, end_date, priority, stackable, is_active	Dynamic discount definitions
`coupon`	**code**, rule_id (FK), max_uses, used_count, expires_on	Promotional voucher codes
`order_discount`	**(order_id, rule_id)** composite PK, amount	Discounts attached to sales
`discount_rule_audit`	**log_id**, rule_id, old_value, new_value, changed_by, changed_at	Rule changes audit log
`employee_audit_log`	**log_id**, employee_id, old_role, new_role, old_branch, new_branch, is_active_changed, changed_by, changed_at	Role/employment changes audit log (Member 5)
`security_event_log`	**log_id**, event_type, details, performed_by, logged_at	Privileged administration audit (Member 5)

---

6. Database programming (The Marking Core)

6.1 Indexing: primary, clustering, and secondary

Primary & Clustered Indexes: In MySQL InnoDB, the primary key **is** the clustered index. Data rows are physically ordered on disk by the PK:

- `order_item(order_id, product_id)` : All items of an order sit contiguously on disk; fetching a receipt is a single sequential range scan.
- `inventory(branch_id, product_id)` : A branch's complete stock is clustered physically together.
- `daily_sales_summary(summary_date, branch_id)` : Date-range analytical scans read contiguous blocks.

Secondary Indexes:

```sql
CREATE INDEX idx_order_date        ON sales_order (order_date);
CREATE INDEX idx_order_branch_date ON sales_order (branch_id, order_date);  -- Composite index
CREATE INDEX idx_order_customer    ON sales_order (customer_id);
CREATE INDEX idx_product_name      ON product (name);
CREATE FULLTEXT INDEX idx_product_ft ON product (name);                     -- Member 3 extra feature
CREATE INDEX idx_order_item_prod   ON order_item (product_id);
```

EXPLAIN Evidence (Member 4):

```sql
EXPLAIN ANALYZE
SELECT branch_id, SUM(total_amount)
FROM sales_order
WHERE branch_id = 2 AND order_date BETWEEN '2025-01-01' AND '2025-12-31 23:59:59'
GROUP BY branch_id;
-- Without index: type=ALL, rows ~ 50,000 (Full Table Scan)
-- With idx_order_branch_date: type=range, Index Range Scan, cost drops by ~95%
```

6.2 Views (Balanced across members)

```sql
-- Member 4: Analytical sales view
CREATE VIEW v_daily_sales AS
SELECT branch_id, DATE(order_date) AS sale_date,
       COUNT(*) AS orders, SUM(total_amount) AS total_sales
FROM sales_order WHERE status = 'COMPLETED'
GROUP BY branch_id, DATE(order_date);

-- Member 2: Supply chain view
CREATE VIEW v_low_stock AS
SELECT i.branch_id, b.name AS branch, p.product_id, p.name AS product,
       i.quantity, i.reorder_level
FROM inventory i
JOIN product p ON p.product_id = i.product_id
JOIN branch b ON b.branch_id = i.branch_id
WHERE i.quantity <= i.reorder_level;

-- Member 3: Customer lifetime value view
CREATE VIEW v_customer_orders AS
SELECT c.customer_id, c.full_name, COUNT(o.order_id) AS orders,
       COALESCE(SUM(o.total_amount), 0) AS total_spent
FROM customer c LEFT JOIN sales_order o
  ON o.customer_id = c.customer_id AND o.status = 'COMPLETED'
GROUP BY c.customer_id, c.full_name;

-- Member 4: Product ranking with Window Function
CREATE VIEW v_product_ranking AS
SELECT p.product_id, p.name, SUM(oi.quantity) AS units_sold,
       SUM(oi.line_total) AS revenue,
       RANK() OVER (ORDER BY SUM(oi.line_total) DESC) AS revenue_rank
FROM order_item oi JOIN product p ON p.product_id = oi.product_id
JOIN sales_order o ON o.order_id = oi.order_id AND o.status = 'COMPLETED'
GROUP BY p.product_id, p.name;

-- Member 3: Active promotional rules
CREATE VIEW v_active_discounts AS
SELECT rule_id, name, rule_type, kind, value, min_order_amount, end_date
FROM discount_rule WHERE is_active AND CURDATE() BETWEEN start_date AND end_date;

-- Member 4: Financial discount impact view
CREATE VIEW v_discount_impact AS
SELECT r.rule_id, r.name, r.rule_type, DATE_FORMAT(o.order_date, '%Y-%m') AS month,
       COUNT(*) AS orders_using, SUM(od.amount) AS total_discount,
       SUM(o.total_amount) AS revenue_after_discount
FROM order_discount od
JOIN discount_rule r ON r.rule_id = od.rule_id
JOIN sales_order o ON o.order_id = od.order_id AND o.status = 'COMPLETED'
GROUP BY r.rule_id, r.name, r.rule_type, DATE_FORMAT(o.order_date, '%Y-%m');

-- Member 5: Dynamic Data Masking View (Protects Customer PII for Cashiers)
CREATE VIEW v_masked_customer_directory AS
SELECT customer_id, full_name,
       fn_mask_phone(phone) AS masked_phone,
       registered_date, loyalty_points
FROM customer;
```

6.3 Functions (UDFs - Guaranteed one per member)

```sql
DELIMITER $$

-- Member 1: Order line item subtotal calculation
CREATE FUNCTION fn_order_total(p_order_id INT) RETURNS DECIMAL(12,2)
READS SQL DATA
BEGIN
  DECLARE v_total DECIMAL(12,2);
  SELECT COALESCE(SUM(line_total), 0) INTO v_total
  FROM order_item WHERE order_id = p_order_id;
  RETURN v_total;
END$$

-- Member 2: Current inventory quantity lookup
```

```
CREATE FUNCTION fn_stock_level(p_branch INT, p_product INT) RETURNS INT
READS SQL DATA
BEGIN
  DECLARE v_qty INT;
  SELECT quantity INTO v_qty FROM inventory
  WHERE branch_id = p_branch AND product_id = p_product;
  RETURN COALESCE(v_qty, 0);
END$$

-- Member 3: Customer loyalty tier classification
CREATE FUNCTION fn_customer_tier(p_customer_id INT) RETURNS VARCHAR(10)
READS SQL DATA
BEGIN
  DECLARE v_spent DECIMAL(14,2);
  SELECT COALESCE(SUM(total_amount), 0) INTO v_spent
  FROM sales_order WHERE customer_id = p_customer_id AND status = 'COMPLETED';
  RETURN CASE WHEN v_spent >= 100000 THEN 'GOLD'
              WHEN v_spent >= 50000  THEN 'SILVER'
              ELSE 'BRONZE' END;
END$$

-- Member 3: Holiday verification
CREATE FUNCTION fn_is_holiday(p_date DATE) RETURNS BOOLEAN READS SQL DATA
BEGIN
  RETURN EXISTS (SELECT 1 FROM holiday WHERE holiday_date = p_date);
END$$

-- Member 3: First purchase eligibility
CREATE FUNCTION fn_is_first_purchase(p_customer INT, p_order INT) RETURNS BOOLEAN READS SQL DATA
BEGIN
  RETURN p_customer IS NOT NULL AND NOT EXISTS (
    SELECT 1 FROM sales_order
    WHERE customer_id = p_customer AND order_id <> p_order AND status = 'COMPLETED');
END$$

-- Member 3: Coupon validity validation
CREATE FUNCTION fn_coupon_valid(p_code VARCHAR(20), p_rule INT) RETURNS BOOLEAN READS SQL DATA
BEGIN
  RETURN p_code IS NOT NULL AND EXISTS (
    SELECT 1 FROM coupon WHERE code = p_code AND rule_id = p_rule
      AND expires_on >= CURDATE() AND used_count < max_uses);
END$$

-- Member 4: Moving average sales metric
CREATE FUNCTION fn_avg_daily_sales(p_branch INT, p_days INT) RETURNS DECIMAL(12,2)
READS SQL DATA
BEGIN
  DECLARE v_avg DECIMAL(12,2);
  SELECT COALESCE(AVG(total_sales), 0) INTO v_avg
  FROM daily_sales_summary
  WHERE branch_id = p_branch AND summary_date >= CURDATE() - INTERVAL p_days DAY;
  RETURN v_avg;
END$$

-- Member 5: Dynamic Data Masking function for PII compliance
CREATE FUNCTION fn_mask_phone(p_phone VARCHAR(20)) RETURNS VARCHAR(20)
DETERMINISTIC
BEGIN
  IF p_phone IS NULL OR LENGTH(p_phone) < 7 THEN
    RETURN p_phone;
  END IF;
  RETURN CONCAT(LEFT(p_phone, 3), '****', RIGHT(p_phone, 3));
END$$

DELIMITER ;
```

6.4 Stored Procedures (With Transactions & Exception Handlers)

```
DELIMITER $$

-- ============================================================================
-- Member 1: Atomic POS Order Placement with Locking & Error Handling
-- ============================================================================
CREATE PROCEDURE sp_place_order(
  IN p_branch INT, IN p_customer INT, IN p_employee INT,
  IN p_method VARCHAR(10), IN p_coupon VARCHAR(20), IN p_items JSON, OUT p_order_id INT)
BEGIN
  DECLARE v_lock INT;
  DECLARE EXIT HANDLER FOR SQLEXCEPTION
  BEGIN
    ROLLBACK;
    RESIGNAL;
  END;

  START TRANSACTION;
  -- Prevent concurrent first-purchase race conditions by locking customer row
  IF p_customer IS NOT NULL THEN
    SELECT customer_id INTO v_lock FROM customer WHERE customer_id = p_customer FOR UPDATE;
  END IF;

  INSERT INTO sales_order (branch_id, customer_id, employee_id)
  VALUES (p_branch, p_customer, p_employee);
  SET p_order_id = LAST_INSERT_ID();

  -- Unpack JSON cart items using MySQL 8 JSON_TABLE; ORDER BY product_id prevents deadlocks
```

```
  INSERT INTO order_item (order_id, product_id, quantity, unit_price)
  SELECT p_order_id, j.product_id, j.qty, p.unit_price
  FROM JSON_TABLE(p_items, '$[*]' COLUMNS (
         product_id INT PATH '$.productId', qty INT PATH '$.qty')) j
  JOIN product p ON p.product_id = j.product_id
  ORDER BY j.product_id;  -- Triggers trg_orderitem_check_stock and reduce_stock fire here

  -- Apply dynamic promotional rules (calls Member 3's cursor procedure)
  CALL sp_apply_discounts(p_order_id, p_coupon);

  -- Record payment
  INSERT INTO payment (order_id, method, amount)
  SELECT p_order_id, p_method, total_amount FROM sales_order WHERE order_id = p_order_id;

  COMMIT;
END$$

-- Member 1: Order Cancellation & Inventory Restoration
CREATE PROCEDURE sp_cancel_order(IN p_order_id INT, IN p_employee INT)
BEGIN
  DECLARE EXIT HANDLER FOR SQLEXCEPTION BEGIN ROLLBACK; RESIGNAL; END;
  START TRANSACTION;
  UPDATE inventory i
  JOIN order_item oi ON oi.product_id = i.product_id AND oi.order_id = p_order_id
  JOIN sales_order o ON o.order_id = p_order_id AND o.branch_id = i.branch_id
  SET i.quantity = i.quantity + oi.quantity;

  UPDATE sales_order SET status = 'CANCELLED' WHERE order_id = p_order_id;
  COMMIT;
END$$

-- ============================================================================
-- Member 2: Restock and Inventory Procedures
-- ============================================================================
CREATE PROCEDURE sp_add_stock(IN p_branch INT, IN p_product INT, IN p_qty INT)
BEGIN
  INSERT INTO inventory (branch_id, product_id, quantity)
  VALUES (p_branch, p_product, p_qty)
  ON DUPLICATE KEY UPDATE quantity = quantity + p_qty;

  INSERT INTO stock_audit_log (branch_id, product_id, change_qty, reason)
  VALUES (p_branch, p_product, p_qty, 'RESTOCK');
END$$

-- Member 2: Inter-Branch Stock Transfer (ACID Transaction)
CREATE PROCEDURE sp_transfer_stock(
  IN p_from_branch INT, IN p_to_branch INT, IN p_product INT, IN p_qty INT)
BEGIN
  DECLARE v_current INT;
  DECLARE EXIT HANDLER FOR SQLEXCEPTION BEGIN ROLLBACK; RESIGNAL; END;

  START TRANSACTION;
  SELECT quantity INTO v_current FROM inventory
  WHERE branch_id = p_from_branch AND product_id = p_product FOR UPDATE;

  IF v_current IS NULL OR v_current < p_qty THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Insufficient stock at source branch';
  END IF;

  UPDATE inventory SET quantity = quantity - p_qty
  WHERE branch_id = p_from_branch AND product_id = p_product;

  INSERT INTO inventory (branch_id, product_id, quantity)
  VALUES (p_to_branch, p_product, p_qty)
  ON DUPLICATE KEY UPDATE quantity = quantity + p_qty;

  INSERT INTO stock_audit_log (branch_id, product_id, change_qty, reason)
  VALUES (p_from_branch, p_product, -p_qty, 'TRANSFER_OUT'),
         (p_to_branch, p_product, p_qty, 'TRANSFER_IN');
  COMMIT;
END$$

-- Member 2: Receive Purchase Order
CREATE PROCEDURE sp_receive_purchase(IN p_po_id INT)
BEGIN
  DECLARE done INT DEFAULT 0;
  DECLARE v_branch INT; DECLARE v_prod INT; DECLARE v_qty INT;
  DECLARE cur CURSOR FOR
    SELECT po.branch_id, poi.product_id, poi.quantity
    FROM purchase_order_item poi
    JOIN purchase_order po ON po.po_id = poi.po_id
    WHERE poi.po_id = p_po_id;
  DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = 1;

  START TRANSACTION;
  UPDATE purchase_order SET status = 'RECEIVED' WHERE po_id = p_po_id;

  OPEN cur;
  item_loop: LOOP
    FETCH cur INTO v_branch, v_prod, v_qty;
    IF done THEN LEAVE item_loop; END IF;
    CALL sp_add_stock(v_branch, v_prod, v_qty);
  END LOOP;
  CLOSE cur;
  COMMIT;
END$$

-- ============================================================================
-- Member 3: Promotional Discount Engine (Cursor Implementation)
```

```sql
-- ============================================================================
CREATE PROCEDURE sp_apply_discounts(IN p_order_id INT, IN p_coupon VARCHAR(20))
BEGIN
  DECLARE done INT DEFAULT 0;
  DECLARE v_rule INT; DECLARE v_type VARCHAR(20); DECLARE v_kind VARCHAR(10);
  DECLARE v_value DECIMAL(10,2); DECLARE v_min DECIMAL(12,2); DECLARE v_max DECIMAL(12,2);
  DECLARE v_tier VARCHAR(10); DECLARE v_stack BOOLEAN;
  DECLARE v_customer INT; DECLARE v_date DATE;
  DECLARE v_subtotal DECIMAL(12,2); DECLARE v_amt DECIMAL(12,2);
  DECLARE v_total DECIMAL(12,2) DEFAULT 0; DECLARE v_ok BOOLEAN;

  DECLARE cur CURSOR FOR
    SELECT rule_id, rule_type, kind, value, min_order_amount, max_discount, tier, stackable
    FROM discount_rule
    WHERE is_active AND CURDATE() BETWEEN start_date AND end_date
    ORDER BY priority;
  DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = 1;

  SELECT customer_id, DATE(order_date), fn_order_total(order_id)
    INTO v_customer, v_date, v_subtotal
  FROM sales_order WHERE order_id = p_order_id;
  SET done = 0;

  OPEN cur;
  rule_loop: LOOP
    FETCH cur INTO v_rule, v_type, v_kind, v_value, v_min, v_max, v_tier, v_stack;
    IF done THEN LEAVE rule_loop; END IF;

    SET v_ok = CASE v_type
      WHEN 'FIRST_PURCHASE' THEN fn_is_first_purchase(v_customer, p_order_id)
      WHEN 'HOLIDAY'        THEN fn_is_holiday(v_date)
      WHEN 'LOYALTY'        THEN v_customer IS NOT NULL AND fn_customer_tier(v_customer) = v_tier
      WHEN 'COUPON'         THEN fn_coupon_valid(p_coupon, v_rule)
      ELSE FALSE END;

    IF COALESCE(v_ok, 0) AND v_subtotal >= v_min AND (v_stack OR v_total = 0) THEN
      SET v_amt = IF(v_kind = 'PERCENT', ROUND(v_subtotal * v_value / 100, 2), v_value);
      IF v_max IS NOT NULL THEN SET v_amt = LEAST(v_amt, v_max); END IF;
      SET v_amt = LEAST(v_amt, v_subtotal * 0.5 - v_total);   -- Capped at 50% max order discount
      IF v_amt > 0 THEN
        INSERT INTO order_discount (order_id, rule_id, amount) VALUES (p_order_id, v_rule, v_amt);
        SET v_total = v_total + v_amt;
        IF v_type = 'COUPON' THEN
          UPDATE coupon SET used_count = used_count + 1 WHERE code = p_coupon;
        END IF;
      END IF;
    END IF;
  END LOOP;
  CLOSE cur;

  UPDATE sales_order
  SET subtotal = v_subtotal, discount_total = v_total, total_amount = v_subtotal - v_total
  WHERE order_id = p_order_id;
END$$

-- ============================================================================
-- Member 4: Reporting Procedure with WITH ROLLUP
-- ============================================================================
CREATE PROCEDURE sp_monthly_report(IN p_year INT, IN p_month INT, IN p_branch INT)
BEGIN
  SELECT DATE(o.order_date) AS day, COUNT(*) AS orders, SUM(o.total_amount) AS sales
  FROM sales_order o
  WHERE o.branch_id = p_branch AND o.status = 'COMPLETED'
    AND YEAR(o.order_date) = p_year AND MONTH(o.order_date) = p_month
  GROUP BY DATE(o.order_date) WITH ROLLUP;
END$$

-- ============================================================================
-- Member 5: Security & Privileged Administration Procedures
-- ============================================================================
CREATE PROCEDURE sp_get_login(IN p_username VARCHAR(50))
BEGIN
  SELECT employee_id, branch_id, role, password_hash
  FROM employee WHERE username = p_username AND is_active = TRUE;
END$$

CREATE PROCEDURE sp_deactivate_employee(IN p_admin_id INT, IN p_target_emp_id INT)
BEGIN
  DECLARE EXIT HANDLER FOR SQLEXCEPTION BEGIN ROLLBACK; RESIGNAL; END;
  IF p_admin_id = p_target_emp_id THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Security Violation: Administrator cannot deactivate own account';
  END IF;

  START TRANSACTION;
  UPDATE employee SET is_active = FALSE WHERE employee_id = p_target_emp_id;
  INSERT INTO security_event_log (event_type, details, performed_by)
  VALUES ('EMPLOYEE_DEACTIVATED', CONCAT('Target Employee ID: ', p_target_emp_id), USER());
  COMMIT;
END$$

CREATE PROCEDURE sp_change_user_role(IN p_emp_id INT, IN p_new_role VARCHAR(20))
BEGIN
  DECLARE EXIT HANDLER FOR SQLEXCEPTION BEGIN ROLLBACK; RESIGNAL; END;
  START TRANSACTION;
  UPDATE employee SET role = p_new_role WHERE employee_id = p_emp_id;
  INSERT INTO security_event_log (event_type, details, performed_by)
  VALUES ('ROLE_CHANGE', CONCAT('Emp ID: ', p_emp_id, ' New Role: ', p_new_role), USER());
  COMMIT;
END$$
```

```
DELIMITER ;
```

6.5 Triggers (Guaranteed ownership per member)

```
DELIMITER $$

-- ============================================================================
-- Member 1: Stock Validation with Row-Level Lock (BEFORE INSERT)
-- ============================================================================
CREATE TRIGGER trg_orderitem_check_stock BEFORE INSERT ON order_item
FOR EACH ROW
BEGIN
  DECLARE v_branch INT; DECLARE v_qty INT;
  SELECT branch_id INTO v_branch FROM sales_order WHERE order_id = NEW.order_id;

  -- Pessimistic locking: locks stock row until transaction commits or rolls back
  SELECT quantity INTO v_qty FROM inventory
   WHERE branch_id = v_branch AND product_id = NEW.product_id
   FOR UPDATE;

  IF v_qty IS NULL OR v_qty < NEW.quantity THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Insufficient stock';
  END IF;
END$$

-- Member 1: Stock Deduction & Audit Log (AFTER INSERT)
CREATE TRIGGER trg_orderitem_reduce_stock AFTER INSERT ON order_item
FOR EACH ROW
BEGIN
  DECLARE v_branch INT;
  SELECT branch_id INTO v_branch FROM sales_order WHERE order_id = NEW.order_id;

  UPDATE inventory SET quantity = quantity - NEW.quantity
   WHERE branch_id = v_branch AND product_id = NEW.product_id;

  INSERT INTO stock_audit_log (branch_id, product_id, change_qty, reason, ref_order_id)
  VALUES (v_branch, NEW.product_id, -NEW.quantity, 'SALE', NEW.order_id);
END$$

-- ============================================================================
-- Member 2: Low-Stock Reorder Alert (AFTER UPDATE)
-- ============================================================================
CREATE TRIGGER trg_inventory_low_stock AFTER UPDATE ON inventory
FOR EACH ROW
BEGIN
  IF NEW.quantity <= NEW.reorder_level AND OLD.quantity > OLD.reorder_level THEN
    INSERT INTO low_stock_alert (branch_id, product_id, quantity)
    VALUES (NEW.branch_id, NEW.product_id, NEW.quantity);
  END IF;
END$$

-- ============================================================================
-- Member 3: Product Price Audit & Rule Bounds Check
-- ============================================================================
CREATE TRIGGER trg_product_price_audit AFTER UPDATE ON product
FOR EACH ROW
BEGIN
  IF OLD.unit_price <> NEW.unit_price THEN
    INSERT INTO price_audit_log (product_id, old_price, new_price, changed_by)
    VALUES (NEW.product_id, OLD.unit_price, NEW.unit_price, USER());
  END IF;
END$$

CREATE TRIGGER trg_rule_check BEFORE INSERT ON discount_rule
FOR EACH ROW
BEGIN
  IF NEW.kind = 'PERCENT' AND NEW.value > 100 THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Percent discount cannot exceed 100';
  END IF;
END$$

-- ============================================================================
-- Member 5: Employee Role & Status Audit (AFTER UPDATE)
-- ============================================================================
CREATE TRIGGER trg_employee_audit AFTER UPDATE ON employee
FOR EACH ROW
BEGIN
  IF OLD.role <> NEW.role OR OLD.branch_id <> NEW.branch_id OR OLD.is_active <> NEW.is_active THEN
    INSERT INTO employee_audit_log (
      employee_id, old_role, new_role, old_branch, new_branch, is_active_changed, changed_by
    ) VALUES (
      NEW.employee_id, OLD.role, NEW.role, OLD.branch_id, NEW.branch_id,
      (OLD.is_active <> NEW.is_active), USER()
    );
  END IF;
END$$

DELIMITER ;
```

6.6 Events (MySQL Event Scheduler)

```sql
SET GLOBAL event_scheduler = ON;

-- Member 4: Precomputed Daily Sales Summary (Nightly at 01:00)
CREATE EVENT ev_daily_sales_summary
ON SCHEDULE EVERY 1 DAY STARTS (TIMESTAMP(CURDATE()) + INTERVAL 1 DAY + INTERVAL 1 HOUR)
DO
  INSERT INTO daily_sales_summary (summary_date, branch_id, total_orders, total_sales)
  SELECT DATE(order_date), branch_id, COUNT(*), SUM(total_amount)
  FROM sales_order
  WHERE status = 'COMPLETED'
    AND order_date >= CURDATE() - INTERVAL 1 DAY AND order_date < CURDATE()
  GROUP BY DATE(order_date), branch_id
  ON DUPLICATE KEY UPDATE total_orders = VALUES(total_orders),
                          total_sales  = VALUES(total_sales);

-- Member 2: Weekly Low-Stock Scan (Weekly)
CREATE EVENT ev_weekly_low_stock_scan
ON SCHEDULE EVERY 1 WEEK
DO
  INSERT INTO low_stock_alert (branch_id, product_id, quantity)
  SELECT i.branch_id, i.product_id, i.quantity FROM inventory i
  WHERE i.quantity <= i.reorder_level
    AND NOT EXISTS (SELECT 1 FROM low_stock_alert a
        WHERE a.branch_id = i.branch_id AND a.product_id = i.product_id AND a.resolved = FALSE);

-- Member 3: Nightly Promotion Expiry
CREATE EVENT ev_expire_discount_rules
ON SCHEDULE EVERY 1 DAY STARTS (TIMESTAMP(CURDATE()) + INTERVAL 1 DAY + INTERVAL 30 MINUTE)
DO UPDATE discount_rule SET is_active = FALSE WHERE end_date < CURDATE() AND is_active;

-- Member 4: Monthly Log Purge (Older than 12 months)
CREATE EVENT ev_monthly_log_purge
ON SCHEDULE EVERY 1 MONTH
DO DELETE FROM stock_audit_log WHERE logged_at < NOW() - INTERVAL 12 MONTH;
```

---

7. Database security and accounts

```sql
-- Dedicated MySQL Users with Least Privilege
CREATE USER 'app_user'@'%'           IDENTIFIED BY 'App_Secret_Pass!2026';  -- Spring Boot API
CREATE USER 'report_user'@'%'        IDENTIFIED BY 'Report_Secret_Pass!2026'; -- Read-only reports
CREATE USER 'etl_user'@'%'           IDENTIFIED BY 'Etl_Secret_Pass!2026';  -- Data warehouse sync
CREATE USER 'admin_user'@'localhost' IDENTIFIED BY 'Admin_Secret_Pass!2026';

-- MySQL 8 Roles (Member 5 Showcase)
CREATE ROLE 'cashier_role', 'manager_role', 'admin_role';

-- Cashier Grants
GRANT EXECUTE ON PROCEDURE supermarket_db.sp_place_order TO 'cashier_role';
GRANT EXECUTE ON PROCEDURE supermarket_db.sp_register_customer TO 'cashier_role';
GRANT SELECT ON supermarket_db.v_masked_customer_directory TO 'cashier_role';
GRANT SELECT ON supermarket_db.v_active_discounts TO 'cashier_role';

-- Manager Grants (Inherits Cashier)
GRANT 'cashier_role' TO 'manager_role';
GRANT EXECUTE ON PROCEDURE supermarket_db.sp_add_stock TO 'manager_role';
GRANT EXECUTE ON PROCEDURE supermarket_db.sp_transfer_stock TO 'manager_role';
GRANT EXECUTE ON PROCEDURE supermarket_db.sp_receive_purchase TO 'manager_role';
GRANT SELECT ON supermarket_db.v_low_stock TO 'manager_role';
GRANT SELECT ON supermarket_db.v_daily_sales TO 'manager_role';
GRANT SELECT ON supermarket_db.v_discount_impact TO 'manager_role';

-- Admin Grants
GRANT ALL ON supermarket_db.* TO 'admin_role';

-- Application user only executes procedures and queries views
GRANT EXECUTE ON supermarket_db.* TO 'app_user'@'%';
GRANT SELECT ON supermarket_db.v_daily_sales TO 'app_user'@'%', 'report_user'@'%';
GRANT SELECT ON supermarket_db.v_low_stock   TO 'app_user'@'%', 'report_user'@'%';
GRANT SELECT ON supermarket_db.v_product_ranking TO 'app_user'@'%', 'report_user'@'%';
GRANT SELECT ON supermarket_db.v_masked_customer_directory TO 'app_user'@'%';
```

---

8. Backend implementation (Spring Boot)

All endpoints under `/api`. No Hibernate, no JPA, no `@Entity`. Data access exclusively uses `JdbcTemplate` and `SimpleJdbcCall`.

Error Mapping:

The global exception handler catches `DataAccessException` and inspects the MySQL error:

- `SQLSTATE 45000` with message `'Insufficient stock'` → **HTTP 409 Conflict**
- Missing or expired JWT → **HTTP 401 Unauthorized**
- Insufficient role permissions → **HTTP 403 Forbidden**
- Validation failure → **HTTP 400 Bad Request**

```
@RestControllerAdvice
public class GlobalExceptionHandler {
  @ExceptionHandler(DataAccessException.class)
  public ResponseEntity<Map<String, String>> handleDbException(DataAccessException ex) {
    String message = ex.getMostSpecificCause().getMessage();
    if (message != null && message.contains("Insufficient stock")) {
      return ResponseEntity.status(HttpStatus.CONFLICT).body(Map.of("error", "Insufficient stock"));
    }
    if (message != null && message.contains("Security Violation")) {
      return ResponseEntity.status(HttpStatus.FORBIDDEN).body(Map.of("error", message));
    }
    return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).body(Map.of("error", "Database error occurred"));
  }
}
```

---

9. Comprehensive 5-Member Work Distribution & Scorecard

9.1 Master Allocation Matrix

Dimension	Member 1 (Sales)	Member 2 (Inventory)	Member 3 (Catalog & Loyalty)	Member 4 (Reporting & Analytics)	Member 5 (Security & Admin)
Domain	Sales, POS & Checkout	Inventory, Restock & Transfers	Catalog, Customers, Discounts	Reports, Analytics & Seed Data	Auth, Security, Audit & DevOps
Tables	`sales_order`, `order_item`, `payment`	`inventory`, `supplier`, `purchase_order`, `purchase_order_item`, `low_stock_alert`	`product`, `category`, `customer`, `discount_rule`, `coupon`, `holiday`, `order_discount`	`daily_sales_summary`, `stock_audit_log` (partitioned)	`employee`, `branch`, `employee_audit_log`, `security_event_log`
Stored Procedures	`sp_place_order` `sp_cancel_order`	`sp_add_stock` `sp_transfer_stock` `sp_receive_purchase`	`sp_save_product` `sp_register_customer` `sp_apply_discounts` (Cursor)	`sp_monthly_report` (`ROLLUP`) `sp_category_sales_cte`	`sp_get_login` `sp_deactivate_employee` `sp_change_user_role`
Triggers	`trg_orderitem_check_stock` `trg_orderitem_reduce_stock`	`trg_inventory_low_stock`	`trg_product_price_audit` `trg_rule_check`	`trg_daily_sales_log`	`trg_employee_audit` `trg_security_alert`
UDF Functions	`fn_order_total`	`fn_stock_level`	`fn_customer_tier` `fn_is_holiday` `fn_coupon_valid`	`fn_avg_daily_sales`	`fn_mask_phone` `fn_check_password_strength`
Views & Events	Clustered composite PK on `order_item`	`v_low_stock` `ev_weekly_low_stock_scan`	`v_active_discounts` `ev_expire_discount_rules`	`v_daily_sales` `v_product_ranking` `v_discount_impact` `ev_daily_sales_summary` `ev_monthly_log_purge`	`v_masked_customer_directory` `v_security_audit_log`
Extra Mark Features	• Concurrency locking (`FOR UPDATE`) • Deadlock retry handling in DAO	• Optimistic locking (`version`) • Atomic inter-branch transfer	• `FULLTEXT` index vs `LIKE` • Priority Cursor discount engine	• Window functions (`RANK`) • Table Partitioning • `EXPLAIN ANALYZE` (50k rows)	• MySQL 8 Roles (`CREATE ROLE`) • Dynamic Data Masking View • `mysqldump` PITR Recovery
Spring Boot Package	`lk.supermarket.sales`	`lk.supermarket.inventory`	`lk.supermarket.product` `customer` `discount`	`lk.supermarket.report`	`lk.supermarket.auth` `audit` `config` `common`
Angular Feature	`features/pos/`	`features/inventory/`	`features/products/` `customers/` `discounts/`	`features/reports/`	`features/auth/` `admin/` `core/` (Guards, Interceptor)
Report Sections (3 each)	2, 6, 13	7, 8, 15	1, 3, 4, 5	12, 14	9, 10, 11

9.2 Viva Demonstration Script per Member (3 minutes each)

1. **Member 1 (Sales / POS)**:
   - Open two MySQL Workbench sessions. Show a product with `quantity = 1`.
   - Start transaction in Session 1, execute sale, do not commit.
   - Run identical sale in Session 2: show session 2 hanging due to `FOR UPDATE`.
   - Commit Session 1; show Session 2 failing with `Insufficient stock`. Stock stays 0.
   - Show POS checkout in Angular, stock reduction, and receipt with applied discounts.
2. **Member 2 (Inventory & Supply Chain)**:

- Call `sp_transfer_stock` from Branch 1 to Branch 2. Show stock decreasing at Branch 1, increasing at Branch 2, and double audit entries in `stock_audit_log`.
- Test transfer with quantity exceeding stock: show rollback and clean error message.
- Trigger `trg_inventory_low_stock` by selling below `reorder_level` and verify alert generated.

3. **Member 3 (Products, Customers & Promotions)**:
   - Demonstrate `FULLTEXT` search index speedup vs `LIKE '%x%'`.
   - Perform price change on a product; show instant audit capture in `price_audit_log`.
   - Demonstrate cursor discount engine: Customer first order earns 10%, adding a holiday date stacks 5%, capping automatically at 50%. Second order loses first-purchase discount.
4. **Member 4 (Reporting & Performance)**:
   - Run `EXPLAIN ANALYZE` on 50,000+ orders demonstrating composite index scan (`idx_order_branch_date`) vs full table scan.
   - Show `v_product_ranking` executing SQL window function `RANK() OVER`.
   - Run `sp_monthly_report` showcasing `WITH ROLLUP` subtotal generation.
   - Show partition pruning on `stock_audit_log` by querying specific year/month.
5. **Member 5 (Security, Roles & Administration)**:
   - Connect as `demo_cashier`: show successful `CALL sp_place_order(...)`, then attempt `DELETE FROM product` and show permission denied error (SQLSTATE 42000).
   - Query `v_masked_customer_directory` as cashier: show masked phone numbers (`077****567`). Query raw table as admin: show full numbers.
   - Execute `sp_deactivate_employee` on an employee and demonstrate `trg_employee_audit` recording the change into `employee_audit_log`.

---

10. Academic Report Sections (15 Sections, 3 per Member)

Section #	Title	Primary Author
1	Introduction & Problem Domain	Member 3
2	Backend Solution & Architecture Overview	Member 1
3	Entity Relationship (ER) Diagram & Cardinality	Member 3
4	Relational Mapping & Normalization (3NF)	Member 3
5	Table Structure, Constraints & Data Types	Member 3
6	Backend Architecture & Stored Procedure Flow	Member 1
7	Tools, Technologies & Platform Choices	Member 2
8	Hosting & Cloud Justification (Local vs Cloud RDS)	Member 2
9	Database Security Measures & RBAC Implementation	Member 5
10	Database User Accounts & Principle of Least Privilege	Member 5
11	Database Connection Management, HikariCP & SSL	Member 5
12	Advanced SQL Code Snippets Compilation	Member 4
13	Problems Encountered & Practical Solutions Log	Member 1
14	Cloud Migration Strategy, High Availability & Parameter Groups	Member 4
15	NoSQL Replacement & Polyglot Persistence Analysis	Member 2

---

11. Final Verification Checklist

- ☐ Strictly no JPA/Hibernate dependencies in `backend/pom.xml`
- ☐ Every table has a PK; composite PK clustering justified in report
- ☐ All 5 team members have at least 1 Stored Procedure with transaction control
- ☐ All 5 team members have at least 1 Trigger enforcing constraints or auditing
- ☐ All 5 team members have at least 1 User-Defined Function (UDF)
- ☐ All 5 team members have at least 1 View
- ☐ 50,000+ dummy orders generated via script for authentic `EXPLAIN` benchmarks
- ☐ MySQL 8 Event Scheduler verified `ON` and scheduled events documented
- ☐ Least-privilege MySQL accounts (`app_user`, `report_user`, `etl_user`, `admin_user`) configured
- ☐ Concurrency locking (`SELECT ... FOR UPDATE`) demonstrated with 2 sessions
- ☐ All SQL scripts numbered in run order (`01_schema.sql` to `10_seed_reference.sql`)
- ☐ Monorepo runnable via `docker compose up --build`